期中习题课：题目与参考答案

题干 · 结论 · 关键解释

依据《期中习题课.pptx》和补充的《作业2 (2).docx》整理，共 7 组、19 题。2.3 原代码取自 DOCX；图片中的表格和伪代码已转写，保留必要条件。参考解法保持简洁，实验数字沿用已验证的实际运行记录。

阅读提示：先看题目，再看紧接的答案；编程题给出关键思路与示意，完整 C 文件和原始日志在配套代码包中。各题的来源按 PPT 文件页序标注。

作业 1：系统调用

题目 1.1

来源：PPT 第 3—4 页。

1.1 Linux 下常见的3种系统调用方法包括：
（1）通过glibc提供的库函数调用
（2）使用syscall函数直接调用相应的系统调用
（3）通过int 80指令（32位系统）或者syscall指令（64位系统）的内联汇编调用
请自行选择Linux系统(kernel>=5.10)的两个系统调用，了解它们的用法，分别使用上述3种系统调用方法来执行。要求：

1. 简述所选择的系统调用的功能；
2. 针对同一个系统调用，记录和对比3种方法的运行时间，并尝试解释时间差异的原因；
3. 对比所选择的两个系统调用在使用内联汇编实现调用时的运行时间，如果有差异，尝试解释差异原因。若无差异，说明即可。

参考答案：选择 getpid 与 getppid。 前者取得当前进程 PID，后者取得父进程 PID。

三种方法分别是 `getpid()`、`syscall(SYS_getpid)`，以及内联汇编执行 x86-64 的 `syscall` 指令；getppid 同理。

本机实测：每组 20 万次，预热后重复 9 次，轮换测试顺序，固定到允许的一个逻辑 CPU，表中为单次调用时间中位数。

系统调用	glibc 封装	syscall()	内联汇编
getpid	75.78 ns	73.65 ns	70.84 ns
getppid	75.61 ns	74.37 ns	74.57 ns

合理解释： 三种方法都需要完成相应内核工作；库封装、通用参数处理和调用约定可能带来少量差异，但路径长短不能单独决定实际耗时。getpid 与 getppid 的内联汇编结果接近，几 ns 差异不足以直接推出普遍的性能高低。原始样本范围见 `results/syscalls.csv`。

现代 glibc 自 2.25 起已取消 getpid 的旧 PID 缓存，不能把现代系统的结果一概解释为缓存命中；某些其他 API（如时钟 API）可能使用 vDSO。原始 syscall 的负错误码与 glibc 的 errno 封装也应区分。

公平测量要点：同一 API／参数、相同循环结构、计时放在循环外、不打印、使用返回值防止优化、预热与重复；反汇编用来检查编译后的真实路径。

源码：`code/syscalls.c`。适用 Linux x86-64；本次为 Linux 6.18.57、glibc 2.41、GCC 14.2.0。

作业 2：程序与进程

题目 2.1

来源：作业2 DOCX 2.1／PPT 第 5—7 页。

2.1 一个C程序可以编译成目标文件或可执行文件。目标文件和可执行文件通常包含text、data、bss、rodata段，程序执行时也会用到堆（heap）和栈（stack）。
（1）请写一个C程序，使其包含data段和bss段，并在运行时包含堆的使用。请说明所写程序中哪些变量在data段、bss段和堆上。
（2）请了解readelf、objdump命令的使用，用这些命令查看（1）中所写程序的data和bss段，截图展示。
（3）请说明（1）中所写程序是否用到了栈。

参考答案：分区示例：

```
int a = 7;                  // .data
int b;                      // .bss
static int c;               // .bss
const char msg[] = "hi";  // 通常在 .rodata

int main(void) {
    int local = 1;          // 本例取地址、关闭优化时位于栈
    int *p = malloc(20);    // p 本身是局部变量；p 指向的 20 字节是动态内存
    /* 使用后 free(p) */
}
```

典型 Linux 工具链把初始化为 0 的全局／静态变量也放入 `.bss`。局部变量不能无条件认定都在栈：优化后可能位于寄存器或被消除。本附例取地址并以 `-O0` 编译，便于观察。

```
readelf -SW code/elf_layout
readelf -sW code/elf_layout
objdump -t code/elf_layout
objdump -s -j .data code/elf_layout
```

`.bss` 是 NOBITS：需要运行时内存并被清零，通常不在文件里保存同等大小的零字节。堆、栈是运行时区域，不应要求在 ELF 节表里找到同名的实际区域。

DOCX 2.1(3)：**用到了栈**。 示例中的局部变量、指针变量及函数调用栈帧使用栈；`malloc` 得到的对象本体在动态分配区域。

PPT 第 7 页的地址对应关系也能解释：装载基址为 `0x55ba6401d000`，符号 `bdata` 的 ELF 地址 `0x4010` 对应运行时 `0x55ba64021010`；`gdata` 的 `0x4060` 对应 `0x55ba64021060`。输出的 `sdata=0x55ba6401f008` 很可能是指针的值，指向 `.rodata` 中偏移 `0x2008` 的字符串；指针变量自身地址应看 `&sdata`，对应 `.data` 中的 `0x4020`。文件偏移不等于虚拟地址。对于 PIE，运行地址通常还要加装载偏移。

源码与证据：`code/elf_layout.c`，`results/elf-layout.txt`、`readelf-*.txt`、`objdump-*.txt`、`elf-evidence.png`。

题目 2.2

来源：作业2 DOCX 2.2／PPT 第 8 页。

2.2 fork、exec、wait等是进程操作的常用API，请调研了解这些API的使用方法。
（1）请写一个C程序，该程序首先创建一个1到42的整数数组，然后创建一个子进程，并让子进程对前述数组所有元素求和，并打印求和结果。等子进程完成求和后，父进程打印“parent process finishes”,再退出。
（2）在（1）所写的程序基础上，当子进程完成数组求和后，让其执行ls -l命令(注：该命令用于显示某个目录下文件和子目录的详细信息)，显示你运行程序所用操作系统的某个目录详情。例如，让子进程执行 ls -l /usr/bin目录，显示/usr/bin目录下的详情。父进程仍然需要等待子进程执行完后打印“parent process finishes”,再退出。
（3）请阅读XV6代码（https://pdos.csail.mit.edu/6.828/2026/xv6.html），找出XV6代码中对进程控制块（PCB）的定义代码，说明其所在的文件，以及当fork执行时，对PCB做了哪些操作?

参考答案：fork／exec／wait：

- 1 到 42 的和为 `42×43/2 = 903`。
- 父进程 `fork`；子进程求和并打印；父进程 `waitpid` 等待，再打印 `parent process finishes`。
- 增加 ls：子进程打印并 `fflush(stdout)` 后执行 `execl("/bin/ls", "ls", "-l", "/usr/bin", (char *)NULL)`。成功的 exec 替换当前进程的程序映像，不新建一个 PID，也不会返回原调用点；失败才返回。父进程等待的是这个子进程最终退出。
- `fork` 后子进程能看到创建前的数组；Linux 通常借助写时复制实现最初的逻辑副本，并不自动让父子修改同一个普通数组。

源码：`code/fork_exec.c`；分别运行 `./fork_exec` 和 `./fork_exec --ls`。运行记录已确认“903 → ls 列表（可选）→ parent process finishes”的顺序。

xv6 的 PCB： `kernel/proc.h` 中的 `struct proc`。包括 PID、状态、父进程、内核栈、页表、trapframe、调度 context、文件表和工作目录等。

官方 `riscv` 分支当前的 fork 内核实现名为 `kfork`（旧版可名为 `fork`），在 `kernel/proc.c`：

1. `allocproc()` 取得空闲 PCB、分配新 PID，准备 trapframe、页表与初始 context。
2. 复制用户地址空间与寄存器现场，设置内存大小。
3. 将子进程的返回寄存器 `a0` 置 0。
4. 对文件、工作目录增加引用，复制名字，设置父进程指针。
5. 将子进程置为 RUNNABLE，向父进程返回子 PID。

这不是直接复制整个 PCB；锁、PID、页表和父子关系须分别初始化。普通 xv6 的 `uvmcopy` 与 Linux 的写时复制实现也不能混为一谈。

题目 2.3

来源：作业2 DOCX 2.3（补全原程序）。

2.3 请阅读以下程序代码，回答下列问题
（1）该程序一共会生成几个子进程？请你画出生成的进程之间的关系（即谁是父进程谁是子进程），并对进程关系进行适当说明。

（2）在不改变for循环的前提下，你能用少量代码修改，使该程序生成多于LOOP个的子进程么？如果可以，也请画出生成的进程之间的关系。

原程序：

```
#include<unistd.h>
#include<stdio.h>
#include<string.h>
#define LOOP 3

int main(int argc,char *argv[])
{
   pid_t pid;
   int loop;

   for(loop=0;loop<LOOP;loop++) {

      if((pid=fork()) < 0)
         fprintf(stderr, "fork failed\n");
      else if(pid == 0) {
         printf(" I am child process\n");
         return 0;
      }
      else {
         sleep(5);
      }
    }
    return 0;
}
```

参考答案：以补充 DOCX 的原代码为准，LOOP=3：

- 原代码子进程打印后 `return 0`，不再进入下一轮：**新建 3 个子进程，总计 4 个进程**，三个孩子都是原父进程直接创建。
- 删除子进程分支中的 `return 0;`，保留循环：每轮存活进程都可能继续 fork，**新建 $2^3-1=7$ 个子进程，总计 8 个进程**；原父进程仍只直接创建 3 个。
- `sleep(5)` 只影响时序，不改变成功 fork 的数量。以下图按逻辑轮次编号，不代表实际 PID 的大小顺序；假设所有 fork 成功。

```
原程序:              删除子进程 return 后:
P0                    P0
├─ P1                 ├─ P1
├─ P2                 │  ├─ P3
└─ P4                 │  │  └─ P7
                      │  └─ P5
                      ├─ P2
                      │  └─ P6
                      └─ P4
```

源码 `code/fork_tree.c` 是带 PID 输出的验证版；`--expand` 对应取消子进程 return，循环和 LOOP 不变。验证版在循环后补了回收子进程的 wait，避免留下僵尸；未增加 fork 调用。两种运行记录实际分别出现 3、7 次 CREATE。

作业 3：线程

题目 3.1

来源：PPT 第 11—13 页。

3.1 pthread_create，pthread_join，pthread_exit
（1）写一个C程序，创建一个值为1到100万的整数数组，求和并计时；
（2）使用pthread函数库创建N个线程（N>1），由这N个线程完成100万个数的求和并计时。
（3）增加绑核操作，将所创建线程和某个CPU核绑定后运行。

参考答案：求和： 1 到 100 万的和是 **500000500000**，必须用 `int64_t`／`long long` 累加，普通 32 位 int 会溢出。

把区间分成 N 段，各线程用局部变量求和，主线程 join 后合并；不能每次加法都抢同一把锁。所有线程应先创建，再统一 join。绑核可用 `pthread_setaffinity_np` 或在线程创建属性中指定；N 个线程都绑同一核不能获得多核并行收益。CPU 编号要从实际允许集合中选。

本次串行约 **0.360 ms**，4 线程不绑核约 **0.510 ms**，含线程创建和 join；这个规模下并行收益没有抵消额外开销。

题目 3.2

来源：PPT 第 14 页。

3.2 写一个 C 程序，首先创建一个有 1 万个元素的整数型随机数组，先使用单进程，然后使用 pthread 创建 N 个线程（N 可以自行决定, 且 N>1），在数组的 1 万个元素中找出最大值并对比两种方法用时。
再将数组大小扩充到1000万做对照试验，对比两组结果

参考答案：求最大值： 每个线程扫描自己的分段，只提交局部最大值，join 后归约。使用相同数组比较，初始化不计入测量。

数组规模	串行中位数	4 线程不绑核	4 线程分散绑核	4 线程同核
1 万	0.005589 ms	0.370238 ms	0.287392 ms	0.274178 ms
1000 万	6.041152 ms	2.339472 ms	2.341882 ms	6.244024 ms

小数组计算量太小，线程管理开销突出；大数组可从多核扫描获益，但还受缓存和内存带宽限制。上述结果不是“多线程必快”或“绑核必快”的定律。所有结果已与串行扫描核对。每个模式预热一次、测 7 次；含线程创建、绑核调用和 join。具体结果会受云环境负载和 CPU 配额影响。

源码：`code/threads.c`。

作业 4：调度

题目 4.1

来源：PPT 第 15 页。

4.1 现有5个作业要在一台计算机上依次执行，它们的运行时间分别是2，6，9，11和X。请问：
1）该以何种顺序运行这5个作业，从而可以获得最短的平均响应时间?
2）如果要获得最短的平均周转时间，该以何种顺序运行这5个作业?

参考答案：两问均用最短作业优先 SJF。 假设所有作业同时到达、单核、非抢占、无切换开销；将 2、6、9、11、X 按升序排列即可，相等者次序任意。响应时间取“到达至首次开始运行”，周转时间取“到达至完成”。

相邻长作业 a 排在短作业 b 前，交换后可减少后者等待且不增加后续作业的总等待。又因为“平均周转时间 = 平均等待时间 + 平均运行时间”，两问的最优顺序相同。

题目 4.2

来源：PPT 第 16 页。

4.2 现有5个作业（作业A、B、C、D、E）同一时间被提交，同时它们的运行时间分别是10、8、4、12和15分钟，计算平均等待时间。
（1）Round robin算法 (使用该算法时，每个作业分到的CPU时间片相等)
（2）优先级调度算法（作业A-E的优先级分别是：2,5,1,3,4，其中5是最高优先级，1是最低优先级）
（3）先到先服务算法 (假设作业的达到顺序是A，B，C，D，E)
（4）最短时间优先算法

参考答案：平均等待时间，单位：分钟： 所有作业 t=0 到达，忽略切换开销；优先级数值越大越优先。

算法	执行顺序／条件	A、B、C、D、E 的等待时间	平均
RR	时间片补充为 3，初始队列 A→B→C→D→E	30、25、18、31、34	**27.6**
优先级	B→E→D→A→C	35、0、45、23、8	**22.2**
FCFS	A→B→C→D→E	0、10、18、22、34	**16.8**
SJF	C→B→A→D→E	12、4、0、22、34	**14.4**

RR 的完成时刻为 A=40、B=33、C=22、D=43、E=49，等待时间 = 完成时刻 − 运行时间。**只说时间片相等而不指定大小，RR 没有唯一数值答案**；27.6 采用 PPT 补充的 3 分钟。

题目 4.3

来源：PPT 第 17—21 页。

4.3 实时系统需要处理两个控制任务：每个任务每 20 ms 执行一次，每次占用 5 ms CPU 时间；同时处理 24 帧/秒的视频，每帧需要 20 ms。该系统是否可调度？若改为 60 帧/秒的新视频，是否仍可调度?

参考答案：实时调度：

```
24 帧/秒: U = 5/20 + 5/20 + 24×20/1000 = 0.98
60 帧/秒: U = 5/20 + 5/20 + 60×20/1000 = 1.70
```

在单核、独立可抢占周期任务、截止时间等于周期、无阻塞与额外开销的模型下，**24 帧可调度，60 帧不可调度**。60 帧按替换原视频理解；若叠加则为 218%，仍不可调度。

EDF 在该模型下的精确利用率判据是 U⩽1。RM 的通用充分界限对 3 任务约为 77.98%，超过它并不说明失败；本题视频的 RM 响应时间迭代为 `20→30→40→40 ms`，小于 `1000/24≈41.667 ms`，所以 RM 也能调度 24 帧。理想最高帧率满足 `0.5+0.02f≤1`，即 **25 帧/秒**；实际需预留开销。

题目 4.4

来源：PPT 第 22 页。

4.4 编写 C 程序，使用 cgroup v2 控制 CPU 利用率：单核上 4 个线程均分总计 50%；两个进程分别占 30% 和 40%。

参考答案：cgroup v2：

目标	子组 cpu.max
同组 4 个线程总上限 50%	`50000 100000`
进程一上限 30%	独立组：`30000 100000`
进程二上限 40%	另一独立组：`40000 100000`

单位是微秒，quota/period 按一个逻辑核的容量计算。4 线程同核、同优先级且持续计算时，每个长期大致 **12.5%**；组配额不保证逐线程精确平分。设置前需有可写的委派父组、启用 `+cpu`，进程 PID 写入对应 `cgroup.procs`；新线程继承进程所属组。

`cpu.max` 是上限，不是保证；`cpu.weight` 是竞争权重，不等于固定百分比。用 `cpu.stat` 的 usage_usec 增量／经过时间和 `top -H`／`pidstat` 验证。

本云环境的 cgroup 目录不可写，**未执行该系统配置实验，也未虚构 30%／40% 的实测结果**。

作业 5：线程同步

题目 5.1

来源：PPT 第 23—26 页。

5.1 编写两线程程序，分别将 1000 万以内的奇数和偶数写入同一数组，共用偏移量 index，保证数据不丢失；完成后输出相邻元素的最大绝对差值。分别实现：
（1）Peterson 算法；（2）pthread_mutex_lock/unlock；（3）__atomic_add_fetch 或 C++ std::atomic::fetch_add。
要求说明临界区；每次进入临界区后执行 200 次操作再退出。

参考答案：关键是分配互不重复的目标位置，同时正确推进各线程自己的数列。 本例明确采用 `0≤数值<10000000`，奇偶各 500 万个；若题目要求 1 到 10000000，可将全部生成值平移 1，差值和同步分析不变。

- 临界区是：使用并更新共享 index、向对应位置写数据。

- Peterson：两个线程的 flag 与 turn 需按正确的原子内存模型实现；仅写 `volatile` 不能解决 C 的数据竞争。本例使用 C11 顺序一致原子操作。
- mutex：锁内连续写 200 个值，每次 `value+=2`；若按元素基数 base 生成，批后应 `base+=200`，不能只加 1。
- GCC 原子分配：`pos=__atomic_add_fetch(&index,1,__ATOMIC_RELAXED)-1`。它返回新值，必须用这次返回值确定位置；C++ `fetch_add` 返回旧值。
- 若要按 200 个为单位分配，可原子加 200，得到独占区间后逐项写入；这表示批量预订位置，不代表把后续写入变成一个不可分割的临界区。
- 主线程必须 join 后再读数组；分配位置数不等于已经写完的位置数。

本例四种模式均实测验证了 1000 万个值完整、无重复、无越界。相邻最大绝对差要扫描最终数组计算，受实际交错影响，**没有一个所有运行都相同的固定答案**。

源码：`code/odd_even.c`（peterson／mutex／atomic1／atomic200）。单次耗时不能直接拿不同批量粒度判定“哪种原语必然更快”。

题目 5.2

来源：PPT 第 27—28 页。

5.2 现有一个长度为5的整数数组，写一个两线程程序，其中线程1负责往数组中写入5个随机数（1到20范围内的随机整数），写完这5个数后，线程2负责从数组中读取这5个数并求和。循环执行5次，每次循环开始时，线程1都重新写入5个数。请思考：
1）上述过程能否通过pthread_mutex_lock/unlock函数实现？如果可以，请写出相应的源代码，并运行程序，打印出每次循环计算的求和值；如果无法实现，请分析并说明原因。

参考答案：只用 mutex 加普通状态变量，可以实现，但需要解锁后轮询。

```
共享 version=0；本地轮次 r=0…4
写线程：持锁且 version==2*r 时，写 5 个数，version++，r++。
读线程：持锁且 version==2*r+1 时，读 5 个数求和，version++，r++。
不轮到自己：解锁再重试，r 不变。
```

锁保护数组和 version，状态保证“写第 r 批→读第 r 批→下一批”。只加锁、不保存轮次，不能保证这种顺序；持锁等待则可能堵住对方。轮询方案消耗 CPU，通常条件变量／信号量更适合，但这题限定不能用它们。

实际 5 轮数据及和已逐项验证：`69、57、38、41、43`，见 `results/alternating.txt`。完整代码 `code/alternating.c`。

题目 5.3

来源：PPT 第 29 页。

5.3 两线程各对 counter 加一 N 次，每次在临界区内再做 L 次计算。分别用 pthread_spin_lock/unlock 与 pthread_mutex_lock/unlock，比较总耗时及 iostat 的 %user、%system，解释两类锁的特性。

参考答案：自旋锁与互斥锁： 两种实现都把 `counter++` 和 L 次附加计算放在同一临界区，每线程执行 N 次，最终必须 `counter=2N`。保持 N、L、绑核、优化选项和计时区间一致，并输出 checksum 防止附加计算被优化掉。

- spin：等待时忙转，消耗 CPU；多核短等待可能更快，同核或持锁者被抢占时可能浪费时间。
- mutex：无竞争通常有用户态快路径，竞争时可阻塞与唤醒；不应认为每次加锁都发生系统调用。
- 本次 24 组运行均验证计数与 checksum。L=500 时，同核 mutex/spin 的墙钟中位数约为 118.8/179.0 ms，分散两核约为 189.5/119.2 ms，说明结论依赖场景。
- `iostat -c -y 1` 观察的是系统总体 %user／%system，不等于某把锁的开销；本环境没有 iostat，附例改用 getrusage 记录进程 CPU 时间，**不能冒充 iostat 实测**。短样本的 CPU 时间尤其不宜过度解释。

源码：`code/locks.c`，参数为 `mutex|spin N L same|spread`。创建成本排除，起始屏障至 join 完成计时；详细记录在 `results/locks.json`。

作业 6：资源与死锁

题目 6.1

来源：PPT 第 30 页。

6.1 某系统存在4个进程和5份可分配资源，当前的资源分配情况和最大需求如下表所示。求满足安全状态下X的最小值。

图中为五类资源，各向量分量对应同一列资源；Available=(0,X,1,0,0)。

进程	Allocated	Maximum
A	(5,4,2,5,3)	(5,5,2,5,5)
B	(3,1,3,2,5)	(3,3,4,2,5)
C	(2,0,3,4,1)	(6,0,6,4,1)
D	(4,2,3,5,2)	(10,2,4,6,11)

参考答案：`Need=Maximum−Allocated`：

进程	Need（按图中五类资源排列）
A	(0,1,0,0,2)
B	(0,2,1,0,0)
C	(4,0,3,0,0)
D	(6,0,1,1,9)

Available=(0,X,1,0,0)。最初只有 B 可能满足需求，必须 X≥2；X=2 时有完整安全序列，故 **最小 X=2**。

```
Work=(0,2,1,0,0)
B 完成 → (3,3,4,2,5)
A 完成 → (8,7,6,7,8)
C 完成 → (10,7,9,11,9)
D 完成 → (14,9,12,16,11)
```

题目 6.2

来源：PPT 第 31 页。

6.2 两进程A和B各需要数据库中的3份记录1、2、3，若进程A以1、2、3的顺序请求这些资源， 进程B也以同样的顺序请求这些资源，则将不会产生死锁。但若进程B以3、2、1的顺序请求这些资源，则可能会产生死锁。这3份资源存在6种可能的请求顺序，其中哪些请求顺序能保证无死锁产生?

参考答案：A 固定按 1→2→3，B 保证不死锁的顺序只有 1→2→3、1→3→2。 两者都先竞争资源 1，胜者可继续取得其余资源并完成，另一方尚未持有资源。

其余四种只是“存在导致死锁的交错”，并非每次运行必死锁。例如 B=2→1→3：A 拿 1，B 拿 2，接着彼此等待。假设每个资源单实例、排他访问，已取得资源在完成前一直持有。本结论已枚举六种顺序验证。

作业 7：高级同步

题目 7.1

来源：PPT 第 32 页。

7.1 设有两个优先级相同的进程T1，T2如下。令信号量S1，S2的初值为 0，已知 z=2，试问T1，T2并发运行结束后 x=? y=? z=?

题中的程序（伪代码）:

```
T1                  T2
x := 1;             y := 1;
x = x + 1;          y = y + 2;
P(S1);              V(S1);
x = x + y;          z := y + 1;
V(S2);              P(S2);
z = x + z;          y = z + y;
```

参考答案：教材模型下，假设每条赋值作为不可分割的语句，最终只有三种结果：

```
(x,y,z) = (5,7,9)、(5,7,4)、(5,12,9)
```

S1 保证 y=3 后 T1 才执行 x=x+y，因此 x=5；S2 保证 T2 的最后一步在 x=5 后，但没有固定三个尾部操作的相对顺序:

剩余关键执行顺序	最终结果
T2:z=y+1 → T2:y=z+y → T1:z=x+z	(5,7,9)
T1:z=x+z → T2:z=y+1 → T2:y=z+y	(5,7,4)
T2:z=y+1 → T1:z=x+z → T2:y=z+y	(5,12,9)

若把赋值拆为多条机器读写操作，结果分析要重新做；若直接写成存在数据竞争的普通 C 线程程序，则不能用这三组值描述标准 C 的全部行为。本答案是课件的语句级模型，已穷举所有允许的交错。

题目 7.2

来源：PPT 第 33 页。

7.2 在生产者-消费者问题中，假设缓冲区大小为5，生产者和消费者在写入和读取数据时都会更新写入/读取的位置offset。请对比分析上述方法一和方法二，哪种方法能让生产者、消费者进程正常运行，并说明分析原因。

初值：empty=5，full=0，mutex=1。图中两种方法按顺序整理如下；“写/读”包含各自的 offset 更新。

方法	生产者 Deposit	消费者 Remove
方法一	P(empty)→P(mutex)→写→V(mutex)→V(full)	P(full)→P(mutex)→读→V(mutex)→V(empty)
方法二	P(mutex)→P(empty)→写→V(full)→V(mutex)	P(mutex)→P(full)→读→V(empty)→V(mutex)

参考答案：方法一的同步顺序正确；方法二可能死锁。

```
初值：empty=5，full=0，mutex=1
生产者：P(empty) → P(mutex) → 放入 → V(mutex) → V(full)
消费者：P(full)  → P(mutex) → 取出 → V(mutex) → V(empty)
```

方法二先取得 mutex：空缓冲区时，消费者可能持 mutex 等 full，生产者却拿不到 mutex，双方不能前进。满缓冲区时也有对称问题。

还要区分同步与下标算法：FIFO 环形队列通常分别维护写指针 in、读指针 out，更新时对 5 取模；单个共享 offset 的加减本身不能证明实现了 FIFO。

题目 7.3

来源：PPT 第 34 页。

7.3 银行有 n 个柜员，每个顾客进入银行后先取一个号，并且等着叫号。当一个柜员空闲后，就叫下一个号。请使用 PV 操作分别实现顾客取号操作 Customer_Service 与柜员服务操作 Teller_Service。

参考答案：一个合理的 PV 实现：FIFO 排队，每个顾客有自己的通知信号量。 假设队列容量足够，n 个柜员并发执行 Teller_Service。

```
共享：queue；qmutex=1；waiting=0；next_ticket=0
每位顾客 c：called[c]=0，arrived[c]=0，finished[c]=0

Customer_Service(c):
    P(qmutex)
    c.ticket = next_ticket++
    queue.push(c)
    V(qmutex)
    V(waiting)
    P(called[c])       // 等待叫到自己，通知不会被别的顾客取走
    走到指定柜台
    V(arrived[c])
    P(finished[c])
    离开

Teller_Service(k):
    while true:
        P(waiting)
        P(qmutex)
        c = queue.pop_front()
        c.counter = k
        叫号(c.ticket) // 在队列锁内记录叫号次序
        V(called[c])
        V(qmutex)
        P(arrived[c])
        为 c 办理业务
        V(finished[c])
```

服务在锁外进行，所以 n 个柜员能并发服务。保证的是取号／叫号 FIFO，不保证各柜员的服务完成顺序。每个顾客专属信号量避免叫号与顾客错配；实际多进程实现须使用进程间共享的同步对象。

题目 7.4

来源：PPT 第 35 页。

7.4 用 pthread 实现 MapReduce：8 个 Map 线程并行读取 8 个文件（每个约 1 MB），分别求局部最大值；1 个 Reduce 线程等全部结果到齐，再求全局最大值。用同步原语协调；若用条件变量，须封装成管程。

参考答案：8 Map + 1 Reduce：完成屏障计数为 9。

```
Map[i]：在锁外读取第 i 个文件 → 求局部最大值
        → 写 local_max[i] → barrier_wait
Reduce：barrier_wait → 求 8 个局部最大值中的最大值
主线程：先创建全部 9 个线程，再 join；不参加这个屏障。
```

另一种答案是管程：submit() 在锁内写结果并 done++；wait_all() 使用 `while(done<8) pthread_cond_wait(...)`，第 8 次提交 signal。读文件和扫描不要放在公共锁内，否则会串行化。

屏障与管程两版已用 8 个各 1 MiB 的二进制 int32 文件运行，局部／全局最大值均与串行扫描相同。Map 文件读取失败时仍提交错误并完成同步，防止其他线程永久等待；已分别做缺文件失败验证。文件格式以示例约定为准。

源码：`code/mapreduce.c`；不要把 PTHREAD_BARRIER_SERIAL_THREAD 的获得者当成固定 Reduce 线程。

使用说明

本资料侧重理解，不把实验耗时当作固定标准答案。完整程序、真实输出与复核脚本见同目录 code、results；4.4 因 cgroup 不可写未执行配置实验，5.3 的 CPU 时间来自 getrusage，未冒充 iostat 数据。

编译：`make -C code -j2`。数学与交错验证：`python3 verify_models.py`。重跑实验：`python3 run_experiments.py`。

环境：Linux 6.18.57、x86-64、glibc 2.41、GCC 14.2.0。原始参考答案与运行记录保留在配套文件夹中。